SmartTask : Context-Aware Mobile To-Do List for Intelligent Task Management

Group ID - 25-26J-531

Line-Up

- Introduction Video
  - Problem & Its Impact
  - Who Is Affected
  - Design Excellence of the Overall Solution
  - Application of Key Knowledge Areas
  - Application of Technologies
  - Closing
- Student's Information
- How the Solution will address the Problem
- User Requirements addressed by the solution
- Project Requirements
- System Design: Six - Component Architecture
- How Good Is The System Design ?
- System Overview: Components Flow In Real World
- How are the risks managed?
- Commercialization/Sustainability of Solution

invideo/smarttask

Problem & Impact

- No Context Awareness
- Static task prioritization
- Poorly timed notifications

Most existing to-do

- frequent
- missed pr
- increased
- reducing

SmartTask : A Context-Aware Mobile Task Management System

IT21228612 | DE SILVA S.J.W.

BSc (Hons) Information Technology Specializing in Software Engineering

How the Solution Addresses the Problem?

Problem

Solution

- **Static task lists →** Tasks are automatically prioritized using real-time user context
- **Poorly timed notifications →** Notifications are delivered only when the user is receptive
- **Frequent interruptions →** Alerts are suppressed during meetings, travel, or focus time
- **High mental effort to manage tasks →** System shows only the most relevant tasks
- **Privacy concerns in cloud-based apps →** All data is processed and stored locally

User Requirements

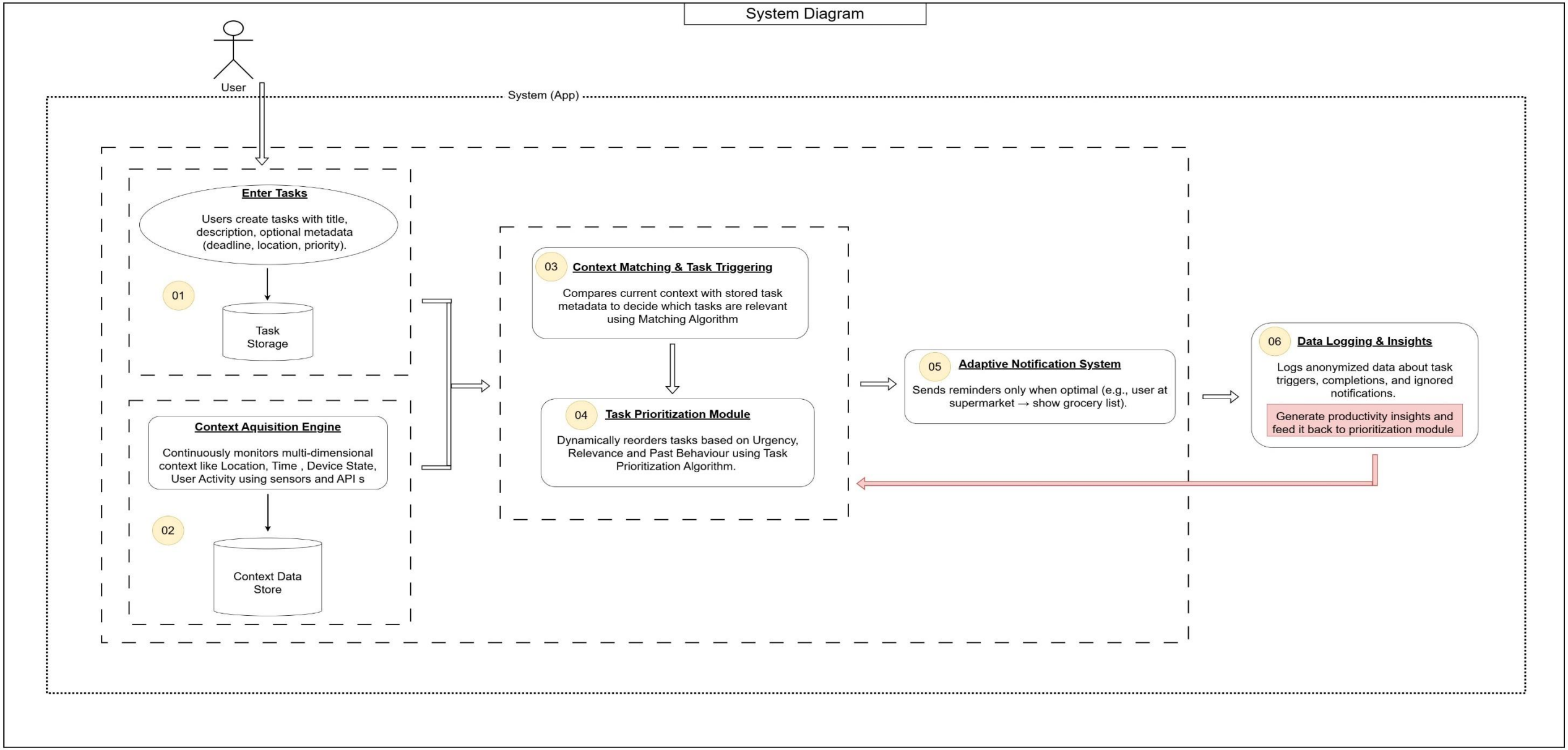
User Requirements Addressed

- Receive task reminders at the right time, based on user context
- Automatically prioritize tasks without manual reordering
- Avoid interruptions during meetings, travel, or focus time
- Privacy ensured with local storage
- Battery saved with less resource usage
- Reduce mental load by showing only relevant tasks

Project Requirements

Requirement Type	Key Requirements
Functional	- Create, edit, delete tasks with metadata (title, deadline, location, priority) - Context acquisition (location, time, device state, user activity) - Real-time context matching & task triggering- Dynamic prioritization (auto + manual) - Adaptive notifications (smart timing, suppression) - Behavioral learning & analytics (completion trends) - Intuitive UI for tasks, lists, and summaries
Non-Functional	- Real-time response (<1s latency) - Minimal, user-friendly interface - Scalable & modular architecture - Reliable with offline support (SQLite/Room) - Data privacy (consent, encryption) - Maintainable code (modular, documented) - Battery & resource efficient
User	- Easy task creation & management - Relevant, non-intrusive notifications - View history & productivity insights - Works offline on Android devices - Control over notifications (quiet hours) - System adapts to personal behavior
System	- Android smartphone (Android 10+, GPS, sensors, 100MB storage) - Development via Android Studio + Java SDK - Local DB (SQLite/Room), Android APIs - Optional ML libs for behavior modeling - GitHub for version control, JUnit/Espresso for testing

System Design: Six - Component Architecture



How Good Is The System Design ?

Design Excellence

- Modular architecture with clear separation of responsibilities
- Context-driven processing pipeline from data capture to decision making
- Offline-first design ensuring privacy and reliability
- Scalable and extensible structure for future learning enhancements
- Efficient background processing aligned with mobile OS constraints

System Overview: Components Flow In Real World

Scenario: A student has 3 upcoming tasks today:

1. Finish Assignment – Deadline: 2:00 PM | Location: Library | Priority: High
2. Buy Stationery – Deadline: Evening | Location: Bookshop | Priority: Medium
3. Call Family – Deadline: Night | Location: Home | Priority: Low

1 - Task Input Module

- Student enters tasks with metadata: title, deadline, location, priority.
- Example: "Finish Assignment – 2:00 PM – Library – High Priority".
- Stored in Task Database.

2 - Context Acquisition Engine

- Continuously gathers real-time signals:
  - Location → detects when student is at home or library.
  - Time → checks system clock (e.g., 1:30 PM, 30 mins before deadline).
  - Device state → battery at 20% triggers early reminder.
  - Activity → detects "walking" → student is moving toward library.
- Stored in Context Data Store.

3 - Context Matching & Task Triggering

- Matches current context with task metadata.
- Example: At 1:30 PM, location = near library, time = 30 mins to deadline.
- Filters task list → highlights "Finish Assignment" as relevant.

4 - Task Prioritization Module

- Orders tasks dynamically:
  - Finish Assignment → top priority (deadline + location match).
  - Buy Stationery → lower (not urgent now).
  - Call Family → lowest (later at night).
- Works in loop with insights to refine ranking.

5 - Adaptive Notification System

- Sends just-in-time reminder:
  - "You're near the library, time to finish your assignment before 2:00 PM."
- Suppresses irrelevant notifications (Call Family won't pop up now).

6 - Data Logging & Insights

- Logs: student opened the reminder, completed assignment at 1:55 PM.
- Stores response + context.
- Feeds back into prioritization → system learns optimal reminder timing (e.g., 30 mins before worked well).

Result:

- Student finishes assignment on time with minimal distractions.
- Notifications felt timely and useful.
- System adapts to user behavior for future days.

Risk Mitigation

How are the risks managed?

- Sensor unavailability → fallback to default task priority
- Permission denial → graceful degradation with warnings
- Battery drain risk → throttled sampling & WorkManager usage
- Notification overload → suppression rules applied

Commercialization Potential

Business Model – How We Make Money

- **Differentiation:** Unlike other apps, ***SmartTask*** is adaptive & context-aware, offering intelligent task prioritization and notifications tuned to real-life conditions.
- **Investment & Cost:**
  - Labor: Development hours for AI algorithms, UI/UX, backend.
  - Material: Android devices for testing.
  - API s : Subscriptions Cost for API s (e.g. google maps).
  - Recovery: Freemium approach – free version with ads; premium subscription offers advanced insights and an ad-free experience.
- **Target Market:** Students, busy professionals, productivity enthusiasts.
- **Customer Profile:** Mobile-first users with heavy task-list dependency and fast-paced lifestyles.
- **Value Proposition:** Reduce stress, improve productivity, and deliver notifications at the right moment, not just the next moment.
- **IP Considerations:** Proprietary algorithms for context-matching and adaptive notification timing create defensible intellectual property.

App Demo

Q & A ??

Thank You !!!